Software Requirements Specification

Unified Service Scheduler

Version	1.2
Date	26 October 2026
Status	Delivered build `service-scheduler` 0.2.0
Readers	The client (dealership and platform owners) and future development teams

1. Introduction

1.1 Purpose

This document states what the Unified Service Scheduler does, the rules it enforces, the quality it guarantees, and how to install and run it. The client can use it to check that the system does what was agreed. A future development team can use it to understand the system before changing it. The design history and the market research behind the decisions are in the separate Technical Document.

1.2 Product in one paragraph

Customers book a service appointment for their motorbike or car at a dealership. The system only offers a time when a **service bay of the right type** and a **technician with the right skill, on shift**, are both free for the **whole job**, and it makes double booking impossible even when many people press "Book" at the same moment. The chosen time is held for 10 minutes while the customer reviews it. Dealership staff run the workshop day, answer advice requests, move jobs between bays and technicians, order and issue replacement parts, and manage bays, technicians, staff accounts and opening hours. A platform manager onboards new dealerships and keeps the service package catalogue.

1.3 Scope

In scope: customer booking of maintenance packages with a 10-minute hold before confirming; advice requests (inspection on request and customisation) that an advisor turns into a bookable job; appointment lifecycle; workshop-day board; workload view and job moves for the workshop controller; replacement parts stock and orders; technician job list; management of bays, technicians, staff and opening hours; the service package catalogue; multi-dealership onboarding; REST API with an OpenAPI contract; logging, metrics, tracing and health checks.

Out of scope for this version: section 12.

1.4 Definitions

Term	Meaning
Dealership	A workshop with its own address, time zone, opening hours, bays and staff. Every business record belongs to one dealership
Bay	A place where a vehicle is worked on. Types: motorbike stand, car lift, diagnostic bay (both vehicle types)

Term	Meaning
Skill	A technician qualification: general servicing, engine & drivetrain, electrical & diagnostics, brakes & suspension, customisation & performance
Service package	A fixed-price job with a checklist, duration, required skill and bay type
Advice request	A customer's description of a problem (inspection) or an idea (customisation). An advisor approves it with a duration, skill, bay type and estimate, or declines it
Slot	A possible start time on the dealership's grid (15, 30 or 60 minutes)
Capacity	The number of (bay, technician) pairs free for the whole job at a slot
D+1	The next day a dealership is open
Idempotency key	A client-chosen header value that makes a repeated booking request return the first result instead of booking again
Hold	A time reserved for one customer for 10 minutes (status HELD) while they review it. It takes the bay, technician and vehicle like a booking. Confirming makes it CONFIRMED; otherwise it becomes EXPIRED and the time is free again
Parts order	A request for replacement parts at one dealership, usually for a booked job: REQUESTED → ORDERED → RECEIVED → ISSUED (or CANCELLED)

1.5 References

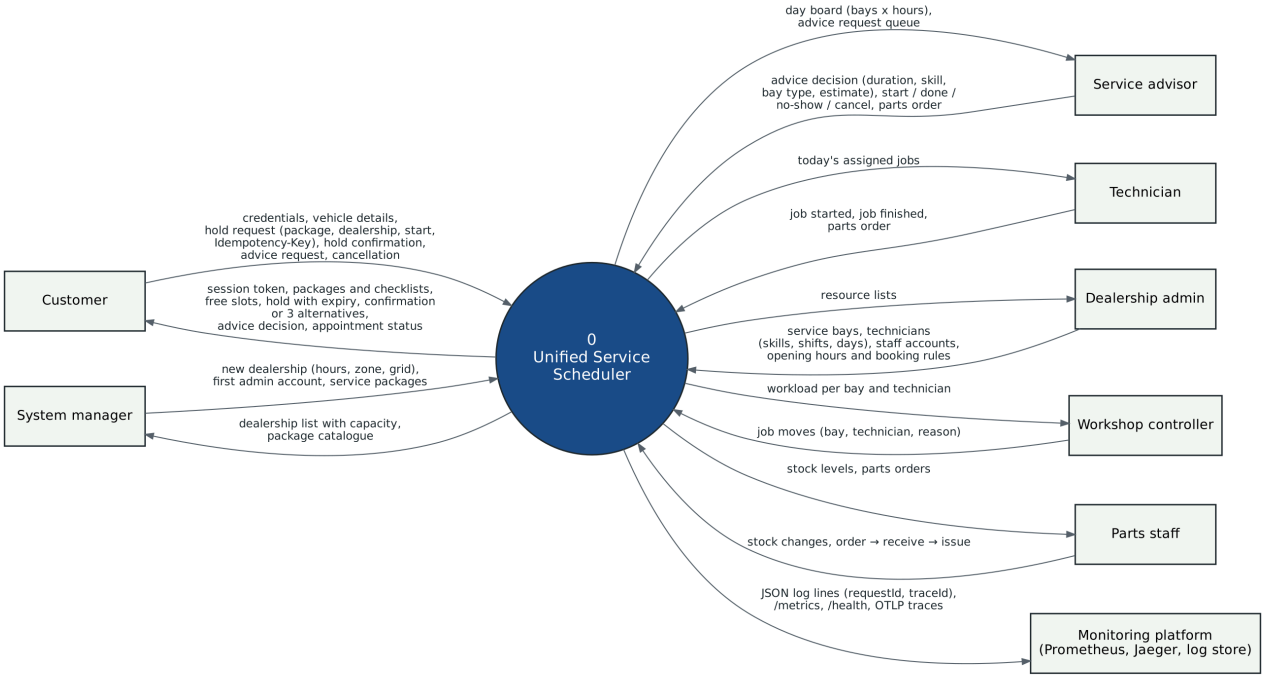
Document	Location
Technical Document	`01-Technical-Document.pdf`
Test Specification	`03-Test-Specification.pdf`
API contract (OpenAPI 3.1)	`backend/conf/openapi.yaml`, served at `/docs`
Database schema	`backend/conf/db/migration/V1__init.sql`
Diagram sources	`diagrams/` (PlantUML and Graphviz)
SOLID review, AI log	`docs/solid-review.md`, `docs/ai-log.md` in the source code

2. Overall description

2.1 Product perspective

A self-contained web application: a React single-page app talks JSON over HTTP to a Scala / Play API, which stores everything in PostgreSQL. It does not depend on any outside service at run time. It plays the role Keyloop's Service Booking and Service Hub play for UK dealerships, and goes further by scheduling bays and technicians rather than advisors only.

Data flow diagram, level 0 (context): Unified Service Scheduler



2.2 User classes

Role	Who	Main goals
Customer	Vehicle owner, often with little technical knowledge	Book a trustworthy slot quickly; ask for advice when unsure what the vehicle needs
Service advisor	Front desk of one dealership	Run the workshop day; turn advice requests into schedulable jobs
Technician	Mechanic at one dealership	See today's jobs and record start and finish
Dealership admin	Manager of one dealership	Everything an advisor does, plus keep bays, technicians, staff and opening hours accurate, move jobs and run the parts shelf
Workshop controller	Plans the workshop floor of one dealership	See how loaded each bay and technician is; move a job when someone is off or a bay is down
Parts staff	Runs the parts counter of one dealership	Keep stock levels right; order, receive and issue the parts that jobs need
System manager	Platform operator	Onboard dealerships, monitor their capacity and keep the service package catalogue

2.3 Operating environment

Component	Requirement
Server	Any OS with JDK 21 or newer (tested on 21 and 25). 1 vCPU and 1 GB RAM are enough for a demo
Database	PostgreSQL 15+ with the `btree_gist` extension. Development and tests use an embedded PostgreSQL 17 started by the API
Browser	Current Chrome, Edge, Firefox or Safari, desktop or phone
Development	sbt 1.10+, Node.js 22+

2.4 Constraints and assumptions

- A job needs exactly one bay and one technician for its whole duration; jobs do not span days.
- Start times sit on the dealership's slot grid inside opening hours; the job must end by closing.
- Bookings open at least 60 minutes ahead and up to 60 days ahead (both configurable per dealership).
- Bay types limit vehicle types: motorbike stand → motorbikes, car lift → cars, diagnostic bay → both.
- Customers book for themselves; staff manage bookings but do not create them in this version.
- A hold lasts 10 minutes (`HOLD_MINUTES`); lapsed holds are released by the next booking that needs the time and by a background sweeper every 30 seconds (`HOLD_SWEEP_SECONDS`).
- Prices are in VND. All times are shown in the dealership's own time zone.

3. Functional requirements

Priority: **M** = must (core of the brief), **S** = should. Status: **Done** = built and covered by an automated test; **Done (UI)** = built and verified in the browser; **Partial** = built in part. Each requirement names its use case (UC) in the use case diagrams of section 9 and its manual test in the Test Specification.

3.1 Accounts and access

ID	Requirement	UC	Priority	Status	Test
FR-01	A visitor can register a customer account with name, email, optional phone and a password of at least 8 characters containing a letter and a digit	UC01	M	Done	TC-AUTH-03
FR-02	A user signs in with email and password and receives a bearer token valid for 12 hours (configurable). A failed sign-in does not reveal whether the email exists	UC02	M	Done	TC-AUTH-01, 02
FR-03	Signing out revokes the token on the server; it cannot be used again	UC02	M	Done	TC-AUTH-04
FR-04	Every API call is checked against the caller's role; staff only see and change data of their own dealership	—	M	Done	TC-AUTH-05, 06, TC-API-01, 02

3.2 Customer

ID	Requirement	UC	Priority	Status	Test
FR-10	A customer can add vehicles (type, make, model, plate, optional year, mileage, 17-character VIN) and list them. A plate is unique per owner	UC03	M	Partial (no edit or delete)	TC-CUS-01
FR-11	A customer can book a maintenance package in four steps: vehicle → package (only packages for that vehicle type) → dealership and time → confirm, with an optional note for the technician	UC04	M	Done	TC-CUS-03

ID	Requirement	UC	Priority	Status	Test
FR-11a	Moving to the confirm step holds the chosen time for 10 minutes with a visible countdown. "Confirm booking" turns the hold into a booking; a hold that ran out is refused (`409 HOLD_EXPIRED`) and the customer can try to hold the time again. Changing the time releases the earlier hold	UC12	S	Done	`FeaturesSpec` › temporary holds; e2e `BookingJourneySpec`
FR-12	The time step shows, for each start time, whether it is free, how many places are left when only one is, and why it is not bookable (full, too soon, not open yet). It opens on the first day that still has a free time and refreshes every 30 seconds	UC20	M	Done	TC-CUS-03
FR-13	If the dealership has no bay or qualified technician for the job at all, the customer is told so and offered no times	UC20	M	Done	TC-CUS-05, TC-SYS-03
FR-14	If the chosen time is taken before the customer could hold it, the hold is refused and the three nearest free times are offered	UC22	M	Done	TC-CUS-04
FR-15	A confirmed booking shows a reference (`SV-XXXXXX`), the technician and the dealership, and can be added to a calendar (.ics)	UC11	S	Done (UI)	TC-CUS-03
FR-16	A customer sees upcoming and past appointments and can cancel one until its start time	UC08, UC09	M	Done	TC-CUS-06
FR-17	A customer can send an advice request (inspection or customisation) for a vehicle at a dealership, choosing the areas concerned and describing it (10–2000 characters)	UC05, UC06	M	Done	TC-CUS-07, 08
FR-18	A customer can withdraw a request that is waiting or approved	UC10	S	Done	TC-CUS-08
FR-19	When an advisor approves a request, the customer sees the reply and the estimate and can pick a time; vehicle, job and dealership are fixed by the request	UC07	M	Done	TC-CUS-09, 10

3.3 Service advisor

ID	Requirement	UC	Priority	Status	Test
FR-30	An advisor sees one day of the dealership as a board (bays × hours) with every job, and can move between days	UC30	M	Done	TC-ADV-01
FR-31	Selecting a job shows its details (customer and phone, vehicle, bay, technician, price, note, checklist) and only the actions its status allows	UC30	M	Done	TC-ADV-02
FR-32	An advisor records that work started (at any time, also before the booked start), that it is finished once the customer has checked the vehicle (the job then awaits payment), or that the customer did not come (only after the start time)	UC32	M	Done	TC-ADV-03

ID	Requirement	UC	Priority	Status	Test
FR-33	An advisor can cancel a booking before work starts	UC33	M	Done	TC-ADV-04
FR-34	An advisor reviews advice requests of the dealership: approve with duration, skill, bay type (compatible with the vehicle), optional estimate and message; or decline with a reason of at least 5 characters	UC31	M	Done	TC-ADV-05, 06
FR-35	An advisor, technician, workshop controller or admin can order replacement parts, optionally for a booked job of the dealership	UC42	S	Done	`FeaturesSpec` › replacement parts; e2e `BookingJourneySpec`

3.4 Technician

ID	Requirement	UC	Priority	Status	Test
FR-40	A technician sees their own jobs for a day, with the checklist and the customer's note	UC40	M	Done	TC-TEC-01
FR-41	A technician starts and finishes their own jobs (finishing puts the job in Awaiting payment); they cannot move another technician's jobs	UC41	M	Done	TC-TEC-02

3.4a Workshop controller

ID	Requirement	UC	Priority	Status	Test
FR-45	A workshop controller (and an advisor or admin) sees, for one day, how many minutes each bay and each technician is booked against their open or shift time	UC34	S	Done	`FeaturesSpec` › workshop controller; `WorkloadSpec`
FR-46	A workshop controller or admin moves a confirmed job to another bay, technician or both, with an optional reason. The new bay must be of the job's type, the technician must have the skill and be on shift for the whole job, and neither may be busy then (`409 RESOURCE_BUSY`). The move is recorded in the audit trail	UC35	S	Done	`FeaturesSpec` › moves a job; never double-books

3.4b Parts staff

ID	Requirement	UC	Priority	Status	Test
FR-47	Parts staff (and admins) add parts (SKU, name, price, stock) and adjust stock; stock can never go below zero	UC70	S	Done	`FeaturesSpec` › keeps the shelf to parts staff and admins
FR-48	Parts staff move an order REQUESTED → ORDERED → RECEIVED → ISSUED, or cancel it. Issuing takes the parts off the shelf; if there is not enough, it is refused (`409 OUT_OF_STOCK`), even when two people issue the last unit at once	UC71	S	Done	`FeaturesSpec` › never issues more than is on the shelf

3.5 Dealership admin

ID	Requirement	UC	Priority	Status	Test
FR-50	An admin has every advisor permission	—	M	Done	TC-ADM-01
FR-51	An admin adds bays and takes them out of service or back; out-of-service bays are never offered	UC50	M	Done	TC-ADM-02
FR-52	An admin adds technicians with skills, shift times, working days and an optional login, and deactivates or reactivates them	UC51	M	Partial (no edit of an existing technician)	TC-ADM-03
FR-53	An admin creates staff accounts (service advisor, admin, workshop controller or parts staff) for their dealership	UC52	S	Partial (no deactivate or password reset)	TC-ADM-04
FR-54	An admin sets opening and closing time, open days, slot size, minimum lead time and booking horizon. New bookings follow them at once; existing bookings are kept and the admin is told how many now fall outside the hours	UC53	S	Done	`FeaturesSpec` › opening hours; e2e `BackOfficeSpec`
FR-55	An admin sees every job of the dealership, newest first, by status (New, In progress, Awaiting payment, Finished, Cancelled & missed) and records that a finished job has been paid, which closes it (Awaiting payment → Finished). Only the admin may close a job; the customer then sees it as Finished. There is no bank integration: the payment is recorded by hand	UC36	M	Done	`ApiSpec` › technician works through it; e2e `WorkshopDaySpec` › records each payment

3.6 System manager

ID	Requirement	UC	Priority	Status	Test
FR-60	A system manager onboards a dealership (code, name, address, IANA time zone, hours, open days) together with its first admin, in one step	UC60	M	Done	TC-SYS-02
FR-61	A system manager sees every dealership with its hours, active bays, active technicians and upcoming appointments (the API also returns the number of staff accounts)	UC61	S	Done	TC-SYS-01
FR-62	A system manager adds service packages (name, vehicle type, price, duration, skill, bay type, checklist), edits them and retires them; customers can book a new package at once	UC62	S	Done	`FeaturesSpec` › service package catalogue; e2e `BackOfficeSpec`

4. Business rules

ID	Rule	Enforced in
BR-01	A bay, a technician and a vehicle can never have two live appointments (Held, Confirmed or In the workshop) whose times overlap. Times are half-open, so 09:00–10:00 and 10:00–11:00 do not overlap	Three PostgreSQL `EXCLUDE USING gist` constraints; also checked in code before inserting
BR-02	A job is only offered when a bay of the required type and a technician with the required skill, on shift on that weekday, are free for the whole duration	`domain/Availability.scala`
BR-03	Best fit: among free technicians, the one with the fewest skills is chosen; ties go to the one with the least booked minutes that day. Specialists stay free for work only they can do	`Availability.allocate`
BR-04	A start time must be on an open day, on the slot grid, inside opening hours, at least `min_lead_minutes` ahead and at most `max_days_ahead` days ahead; the job must end by closing	`Availability.validateStart` (called by `BookingService`), the same rules in the slot grid
BR-05	Lifecycle: Held → Confirmed (by the customer, before the hold runs out); Held → Cancelled (released) or Expired (ran out); Confirmed → In the workshop (at any time: the vehicle may arrive early) → Awaiting payment → Finished; Confirmed → Cancelled (before work starts); Confirmed → Missed (no-show, only after the start). No way back from Finished, Cancelled, Missed or Expired	`domain/Lifecycle.scala`, `AppointmentService.confirmHold`
BR-06	A customer may only confirm and cancel; a technician may only start and finish their own jobs; advisors may start, finish, cancel and record a no-show at their dealership; only the admin records a payment (Awaiting payment → Finished); workshop controllers, parts staff and the system manager may not change a status	`Lifecycle.roleMayMoveTo`, `AppointmentService`
BR-07	An appointment is either for a package or for an approved advice request, never both. One advice request has at most one live booking	Database `CHECK` and partial unique index
BR-08	Cancelling the appointment of an advice request, or its hold expiring, re-opens the request, so the customer can pick another time	`AppointmentService`, `HoldSweeper`, `expireHolds`
BR-09	An approved advice request must carry duration, skill and bay type, and the bay type must accept the vehicle type	Database `CHECK`; `ConsultationService.review`
BR-10	The same `Idempotency-Key` from the same customer never creates a second appointment; the repeat returns the first one	Unique index `(customer_id, idempotency_key)` and an advisory lock
BR-11	Every status change and every job move is recorded with who, when, from and to	`appointment_events`, `part_order_events`
BR-12	A moved job keeps every booking rule: bay of the job's type, technician with the skill and on shift for the whole job, no overlap	`DispatchService.reassign`, the BR-01 constraints
BR-13	Parts stock never goes below zero; a parts order line uses a part of the order's own dealership	`CHECK (stock_qty >= 0)`, composite foreign key, row locks in `PgPartsRepo`

5. Non-functional requirements

ID	Area	Requirement	How it is met and verified
NFR-01	Correctness under concurrency	Any number of simultaneous bookings for one slot produce exactly as many appointments as there is capacity, and never an error	`BookingConcurrencySpec` (50 at once); load test: 1000 at once → exactly 4 × 201 and 996 × 409, 0 errors
NFR-02	Throughput	One instance on 2 shared vCPUs answers a burst of 1000 simultaneous requests without errors	Load test: 1000 bookings spread over 54 slots in 3.2 s; 1000 availability reads in 2.1 s
NFR-03	Scalability	The API keeps no state in memory, so more instances can run behind a load balancer	Sessions and locks live in PostgreSQL; BR-01 holds across instances
NFR-04	Security	Passwords hashed with PBKDF2-HMAC-SHA512 and a per-user salt; bearer tokens stored only as SHA-256 hashes; tokens revocable; role and dealership checked on every call; no secrets in logs	`Passwords.scala`, `ApiSpec` role and isolation tests
NFR-05	Reliability	Deadlocks and lost races are retried (up to 5 attempts); transient database errors answer `503` with `Retry-After`	`BookingServiceFakesSpec`, error handler
NFR-06	Observability	One JSON log line per event with request ID, trace ID, user, role and dealership; Prometheus metrics; OpenTelemetry traces (request, booking attempt and SQL spans, exported over OTLP); health check; slow-query log over 200 ms	`/metrics`, `/health`, TC-OPS-01 to 04, `FeaturesSpec` › tracing
NFR-07	Traceability	Every response and error carries an `X-Request-Id`; the UI shows it for unexpected errors	`ApiSpec` › request-ID propagation
NFR-08	Maintainability	Hexagonal modules; inner layers never import adapters, HTTP, JDBC or Anorm; compiles with `-Wunused:all -Werror`	`ArchitectureSpec`
NFR-09	Testability	Backend unit, property and integration tests on the real database engine (including property-based tests over thousands of random workshops), plus Selenium end-to-end tests in Chrome on every pull request	`sbt test` in `backend/`, `sbt test` in `e2e/`
NFR-10	Usability	Customer flow works on phones and desktops; availability opens on the first free day; plain-language errors	Verified in a headless browser at desktop and phone sizes
NFR-11	Time zones	Every dealership has its own IANA time zone; slot times stay correct across daylight-saving changes	`AvailabilitySpec` (Europe/London)
NFR-12	API contract	Every route is documented in OpenAPI 3.1	`ApiSpec` fails if a route is missing from `openapi.yaml`

6. External interfaces

6.1 User interface

Screen	Path	Roles
Sign in, Create account	`/login`, `/register`	Anyone
Home	`/`	Customer
Book a service	`/book`, `/book?request=<id>`	Customer
My appointments	`/appointments`	Customer
Advice requests, new request	`/requests`, `/requests/new?kind=INSPECTION	MODIFICATION`
My vehicles	`/vehicles`	Customer
Workshop day	`/staff/board`	Service advisor, admin, workshop controller
Workload	`/staff/workload`	Service advisor, admin, workshop controller
Parts	`/parts`	Service advisor, technician, admin, workshop controller, parts staff
Advice requests (staff)	`/staff/requests`	Service advisor, admin
Bays & team	`/admin`	Admin
My jobs	`/jobs`	Technician
Dealerships	`/system`	System manager
Service packages	`/system/packages`	System manager

Screenshots of every screen are in the Test Specification.

6.2 REST API

JSON over HTTP. Authentication: `Authorization: Bearer <token>` from `POST /api/auth/login`. Errors always have the shape `{"error":{"code","message","details?","requestId"}}`. Times are ISO-8601 with offset. The full contract with request and response schemas is served at `/docs`.

Method	Path	Purpose	Roles
POST	`/api/auth/register`	Create a customer account and sign in	Anyone
POST	`/api/auth/login`	Sign in	Anyone
POST	`/api/auth/logout`	Revoke the token	Signed in
GET	`/api/auth/me`	The signed-in user	Signed in
GET	`/api/dealerships`	Dealerships to book with	Signed in
GET	`/api/service-packages`	Maintenance packages	Signed in
GET	`/api/reference-data`	Inspection areas, skills, bay types	Signed in
GET, POST	`/api/me/vehicles`	List or add own vehicles	Customer

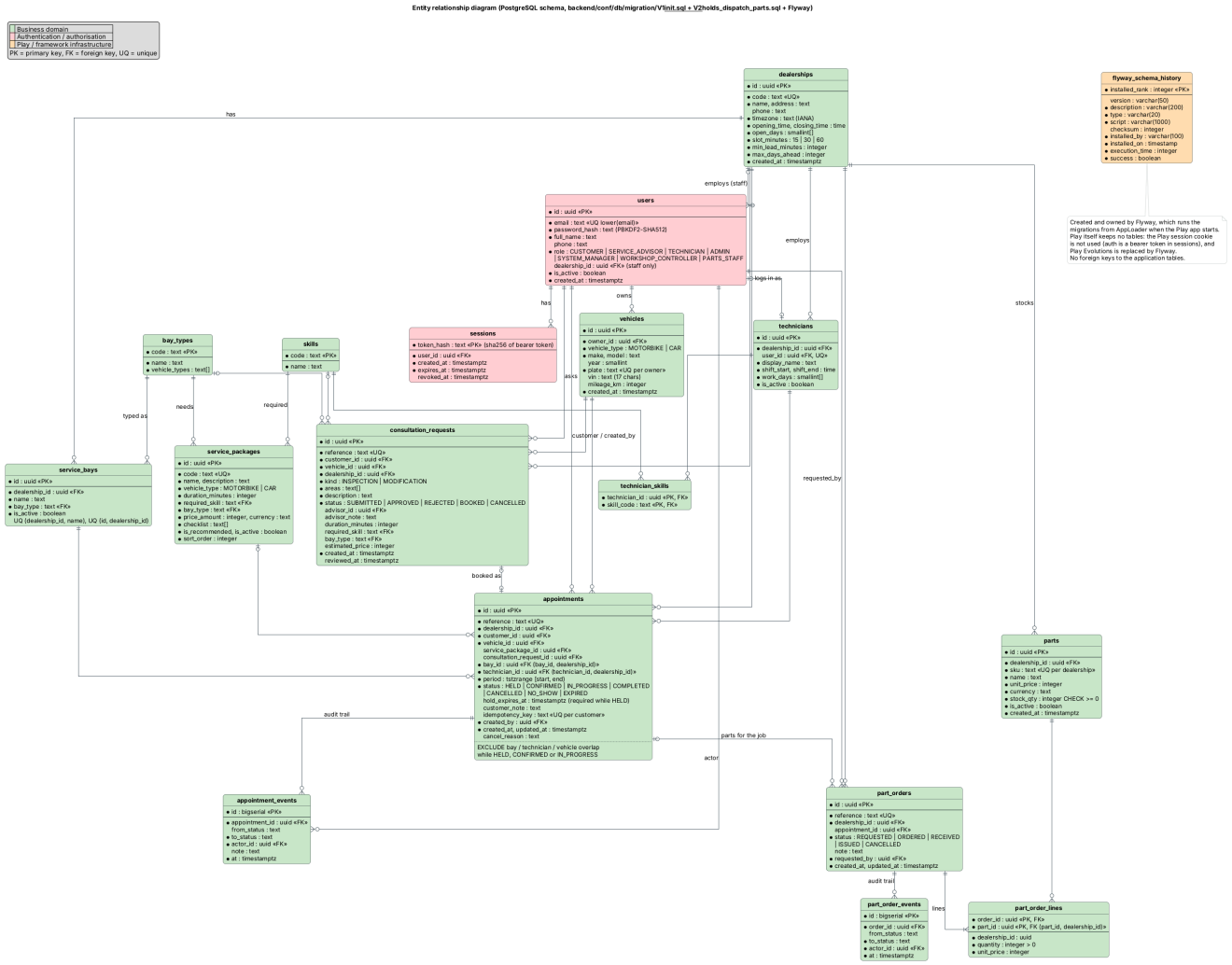
Method	Path	Purpose	Roles
GET	`/api/availability`	Slot grid for a dealership, date and package or approved request	Customer, advisor, admin
POST	`/api/appointments`	Book straight to CONFIRMED (optional `Idempotency-Key`)	Customer
POST	`/api/appointments/holds`	Hold a time for 10 minutes (optional `Idempotency-Key`)	Customer
POST	`/api/appointments/:id/confirm`	Confirm a hold	Customer (own)
POST	`/api/appointments/:id/reassign`	Move a job to another bay and/or technician	Workshop controller, admin
GET	`/api/me/appointments`	Own appointments	Customer
GET	`/api/appointments/:id`	One appointment	Owner, staff of its dealership
POST	`/api/appointments/:id/cancel`	Cancel	Customer (own, before start), advisor, admin
POST	`/api/appointments/:id/status`	Start, finish (→ awaiting payment), no-show; payment received (→ completed) by the admin only	Advisor, admin, technician (own)
GET	`/api/staff/appointments`	One day at the dealership	Advisor, admin, workshop controller
GET	`/api/staff/workload`	Booked minutes per bay and technician for a day	Workshop controller, advisor, admin
GET	`/api/staff/resources`	Active bays and technicians	Advisor, admin, workshop controller
GET	`/api/technician/jobs`	Own jobs for a day	Technician
POST	`/api/consultations`	Send an advice request	Customer
GET	`/api/me/consultations`	Own advice requests	Customer
GET	`/api/consultations/:id`	One advice request	Owner, staff of its dealership
POST	`/api/consultations/:id/cancel`	Withdraw	Customer (own)
GET	`/api/staff/consultations`	The dealership's advice requests	Advisor, admin
POST	`/api/consultations/:id/review`	Approve or decline	Advisor, admin
GET	`/api/admin/jobs`	Every job of the dealership, newest first, optionally one `status`	Admin
GET, POST	`/api/admin/bays`	List or add bays	Admin
PATCH	`/api/admin/bays/:id`	Put in or out of service	Admin
GET, POST	`/api/admin/technicians`	List or add technicians	Admin
PATCH	`/api/admin/technicians/:id`	Activate or deactivate	Admin
GET, POST	`/api/admin/staff`	List or add staff accounts	Admin

Method	Path	Purpose	Roles
PUT	`/api/admin/opening-hours`	Opening hours and booking rules	Admin
GET, POST	`/api/system/dealerships`	List or onboard dealerships	System manager
GET, POST	`/api/system/packages`	List or add service packages	System manager
PATCH	`/api/system/packages/:id`	Edit or retire a package	System manager
GET	`/api/parts`	The dealership's parts and stock	Advisor, technician, admin, workshop controller, parts staff
POST	`/api/parts`, `/api/parts/:id/stock`	Add a part, adjust stock	Parts staff, admin
GET, POST	`/api/part-orders`	List or raise parts orders	Advisor, technician, admin, workshop controller, parts staff
POST	`/api/part-orders/:id/status`	Order, receive, issue or cancel	Parts staff, admin
GET	`/health`	Database check	Anyone
GET	`/metrics`	Prometheus metrics	Operator
GET	`/docs`, `/docs/openapi.yaml`	Swagger UI and the contract	Anyone

Main status codes: `201` created, `200` idempotent replay, `401` not signed in, `403` wrong role or dealership, `404` not found or not yours, `409 SLOT_UNAVAILABLE` (with `details.alternatives`), `409 TOO_LATE_TO_CANCEL`, `409 HOLD_EXPIRED`, `409 RESOURCE_BUSY`, `409 OUT_OF_STOCK`, `422` validation (`NOT_ON_SLOT_GRID`, `TOO_SOON`, `NOT_OFFERED_HERE`, `PACKAGE_NOT_FOR_VEHICLE`, …), `503 TRY_AGAIN`.

7. Data model

The schema has 18 tables in three groups (migrations `V1__init.sql` and `V2__holds_dispatch_parts.sql`). The entity relationship diagram colours them: **green** for the business domain, **red** for authentication and authorisation, **orange** for framework infrastructure.



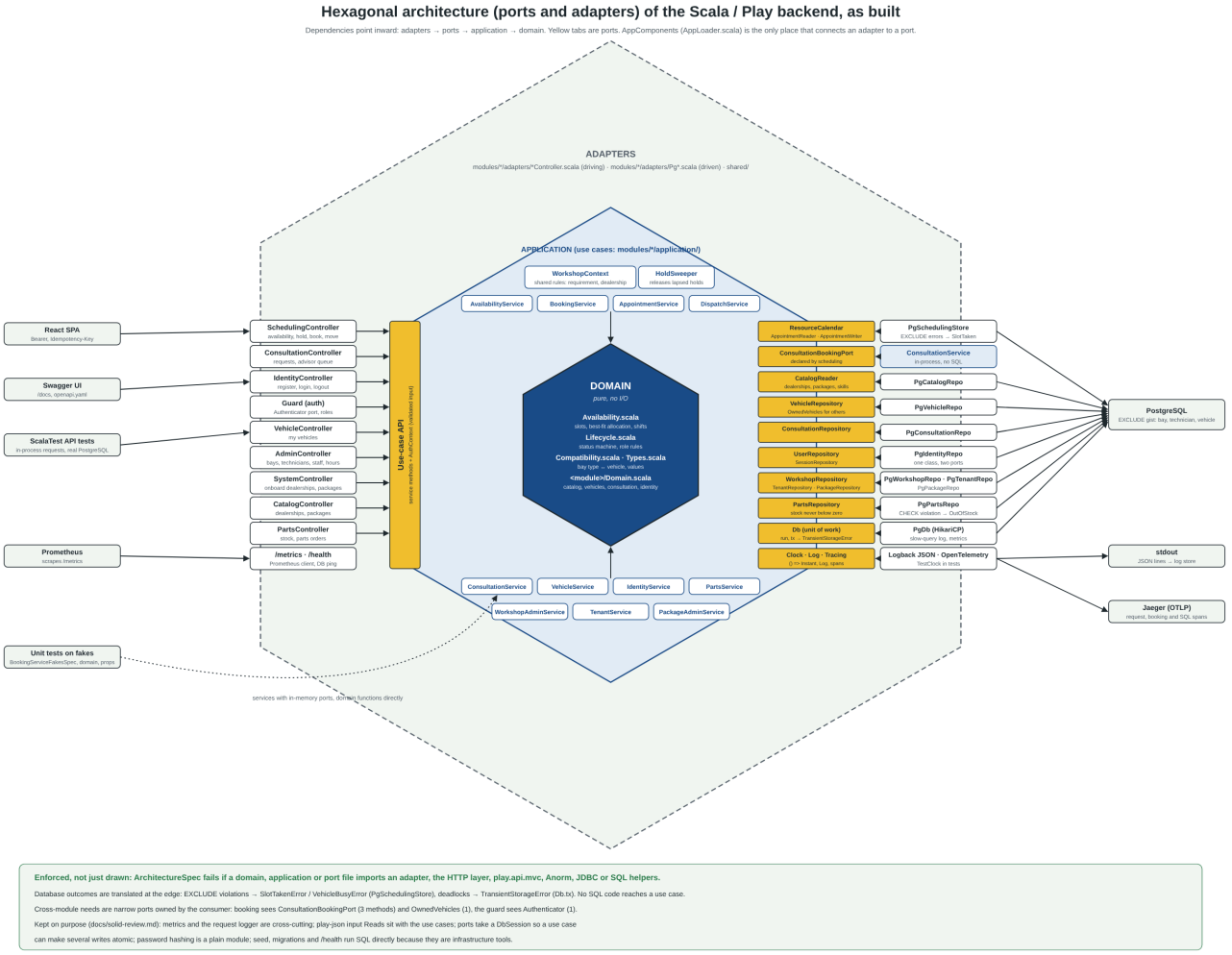
Group	Table	Holds
Business domain	`dealerships`	Tenants: code, address, time zone, hours, open days, slot size, lead time, booking horizon
Business domain	`skills`, `bay_types`	Reference data; a bay type lists the vehicle types it accepts
Business domain	`service_bays`	Bays per dealership, with type and in-service flag
Business domain	`technicians`, `technician_skills`	Technicians per dealership, shift, working days, skills, optional login
Business domain	`service_packages`	The catalogue: price, duration, skill, bay type, checklist
Business domain	`vehicles`	Customers' vehicles
Business domain	`consultation_requests`	Advice requests and the advisor's decision
Business domain	`appointments`	Bookings and holds, with the period as `tstzrange`, `hold_expires_at`, and the three exclusion constraints (HELD, CONFIRMED, IN_PROGRESS)
Business domain	`appointment_events`	Audit trail of every status change and job move
Business domain	`parts`	Parts stock per dealership; `stock_qty` can never be negative
Business domain	`part_orders`, `part_order_lines`	Parts orders, optionally for an appointment, and their lines
Business domain	`part_order_events`	Audit trail of parts order status changes

Group	Table	Holds
Authentication and authorisation	`users`	Every account, its role and (for staff) its dealership; password hash
Authentication and authorisation	`sessions`	SHA-256 hashes of bearer tokens, expiry, revocation
Framework infrastructure	`flyway_schema_history`	Applied migrations, written by Flyway when the Play application starts

Play Framework itself creates no tables: the application does not use Play's session cookie (it uses bearer tokens stored in `sessions`), and schema changes are made by Flyway instead of Play Evolutions. `flyway_schema_history` is therefore the only framework table.

8. System architecture

Hexagonal (ports and adapters) inside a modular monolith. Each module has the same layers: pure domain, application services that depend only on ports (traits), and adapters for HTTP and PostgreSQL. `AppLoader` is the only place that decides which adapter implements which port.



Module	Responsibility
`scheduling`	Availability, holds and booking, appointment lifecycle, workload and job moves, the hold sweeper
`consultation`	Advice requests and their review

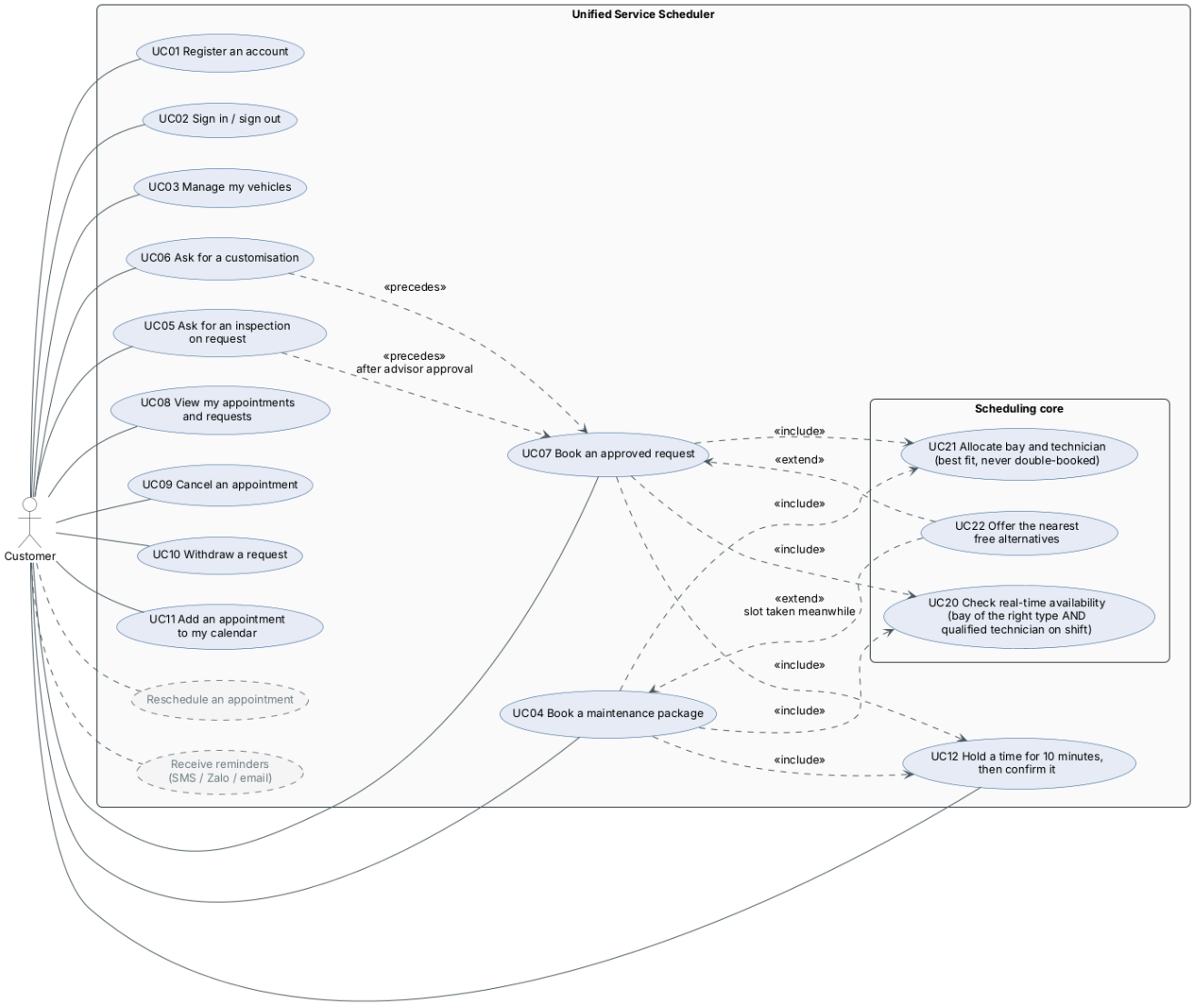
Module	Responsibility
`vehicles`	Customers' vehicles
`catalog`	Dealerships, packages, reference data
`identity`	Accounts, passwords, sessions, the auth guard
`admin`	Bays, technicians, staff and opening hours of a dealership
`parts`	Parts stock and parts orders
`system`	Dealership onboarding and monitoring, the service package catalogue
`shared`	Configuration, logging, errors, database port and pool, metrics, migrations
`http`	Request IDs, trace spans, error format, request logging, `/health`, `/metrics`, `/docs`

9. Diagrams

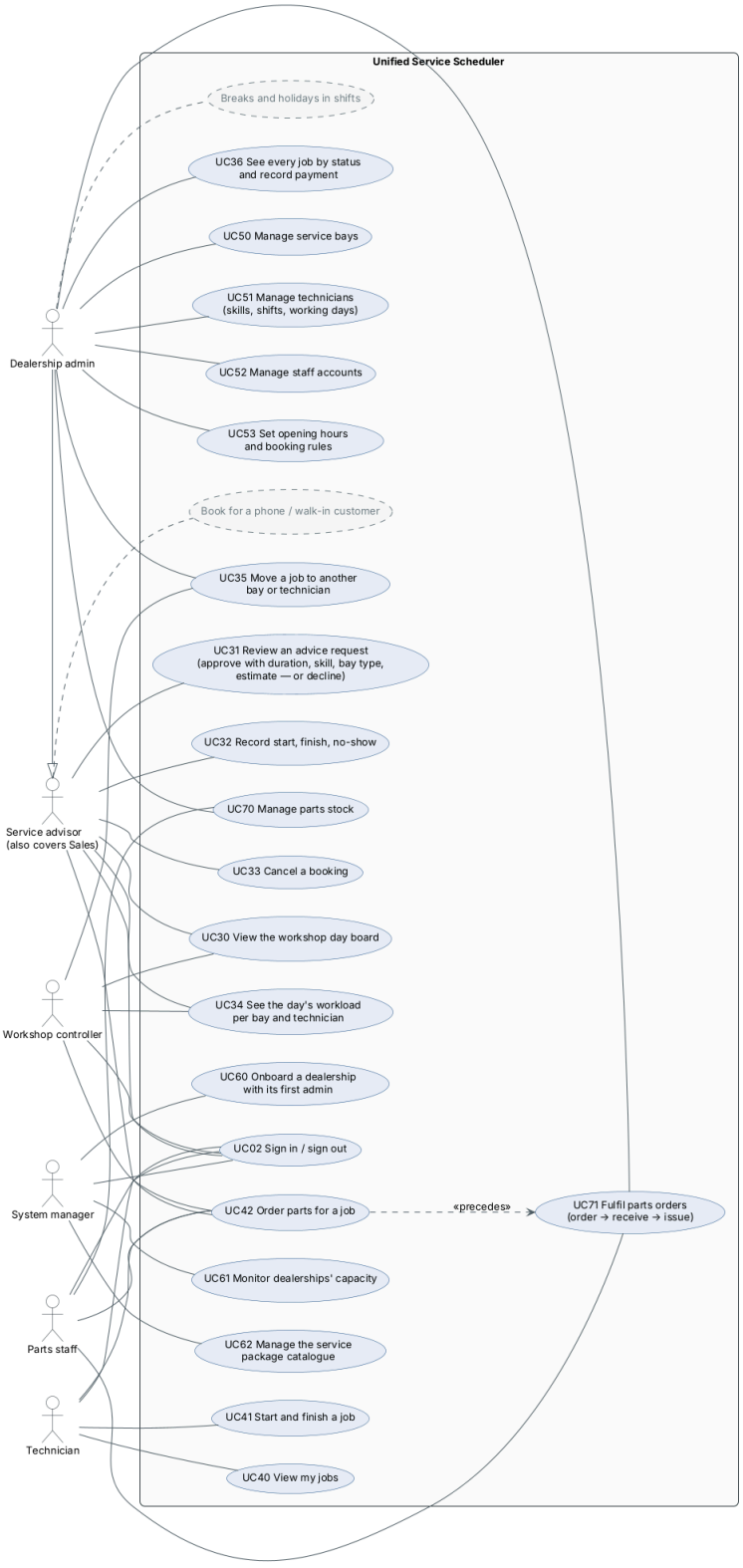
All diagrams are in the `diagrams` folder as PNG and SVG, with their PlantUML (`.puml`) or Graphviz (`.dot`) sources. Solid shapes are built; dashed grey shapes are future work.

#	Diagram	File	Shows
1	Use cases, customer	`use-case-customer.png`	UC01–UC12, UC20–UC22
2	Use cases, staff	`use-case-staff.png`	UC30–UC71 for advisor, technician, workshop controller, parts staff, admin, system manager
3	Activity, book a maintenance package	`activity-book-package.png`	The main customer flow with its checks, the hold and the confirm
4	Activity, inspection or customisation	`activity-consult-first.png`	Consult first, then book
5	Activity, the workshop day	`activity-workshop-day.png`	Workshop controller, advisor and technician during the day
6	State, appointment lifecycle	`state-appointment.png`	Statuses and who may move between them
7	Data flow, level 0	`dfd-context.png`	The system and its external entities
8	Data flow, level 1	`dfd-level1.png`	Nine processes and nine data stores
9	Class, domain model	`class-domain-model.png`	Persisted entities with multiplicities
10	Class, application layer	`class-application.png`	Services, ports and their PostgreSQL adapters
11	Hexagonal architecture	`hexagonal-architecture.png`	Modules, ports and adapters as built
12	Entity relationship	`erd.png`	All 18 tables, colour-coded by group

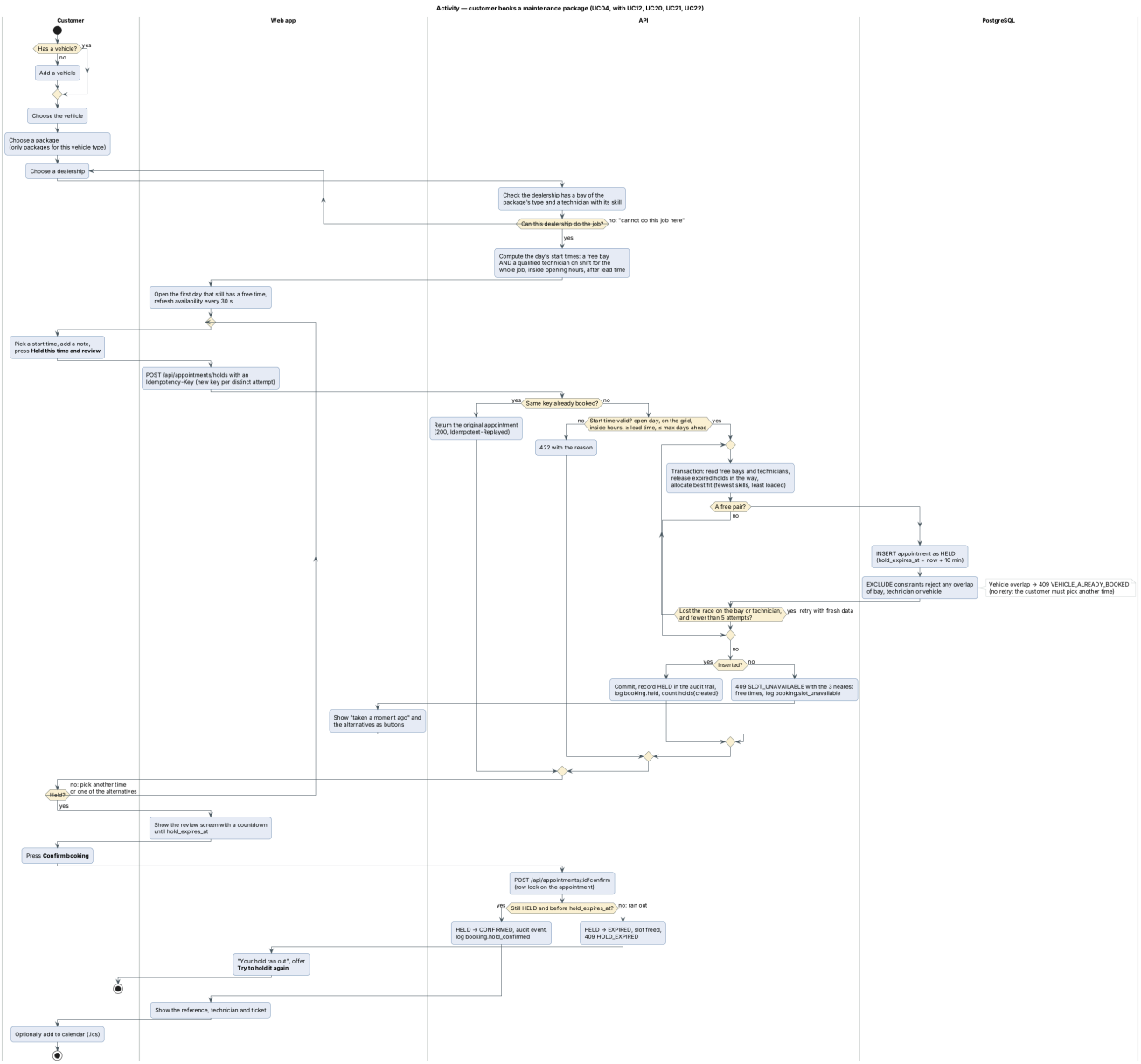
Use cases — customer

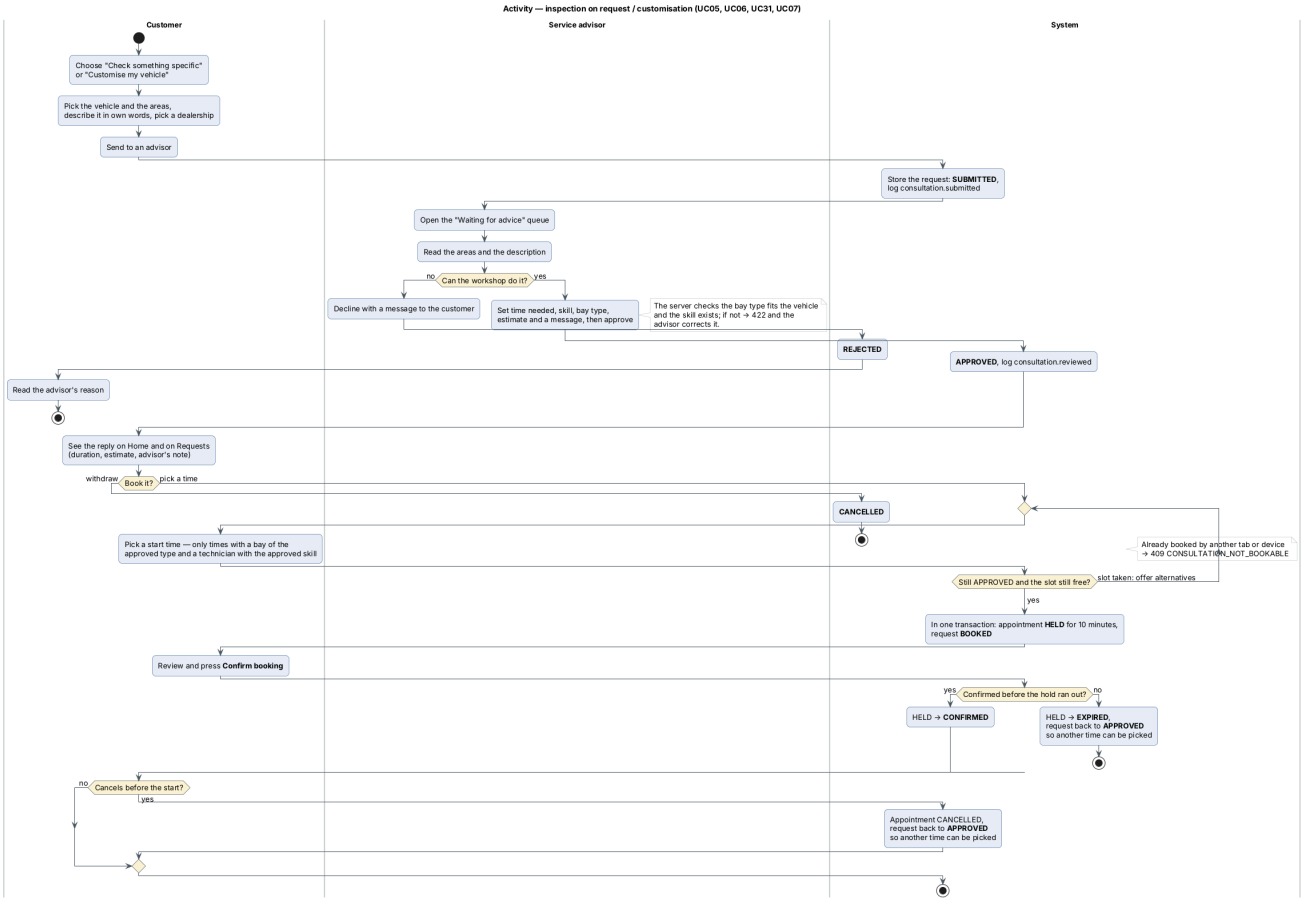


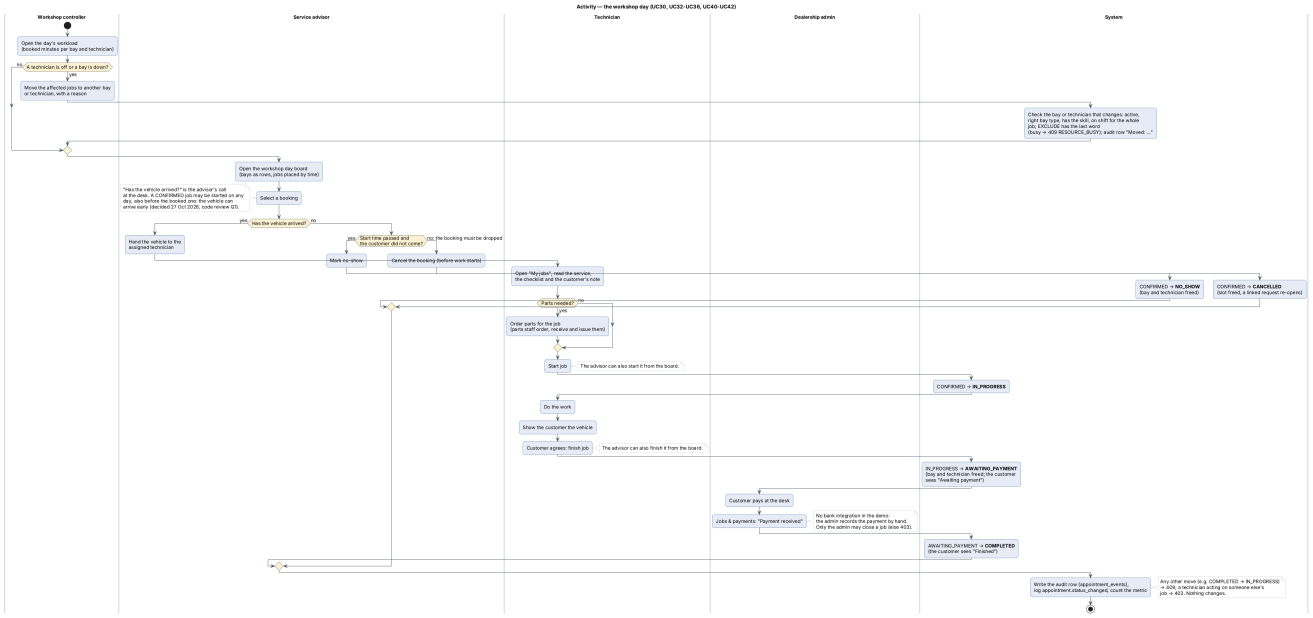
Use cases — dealership and platform staff

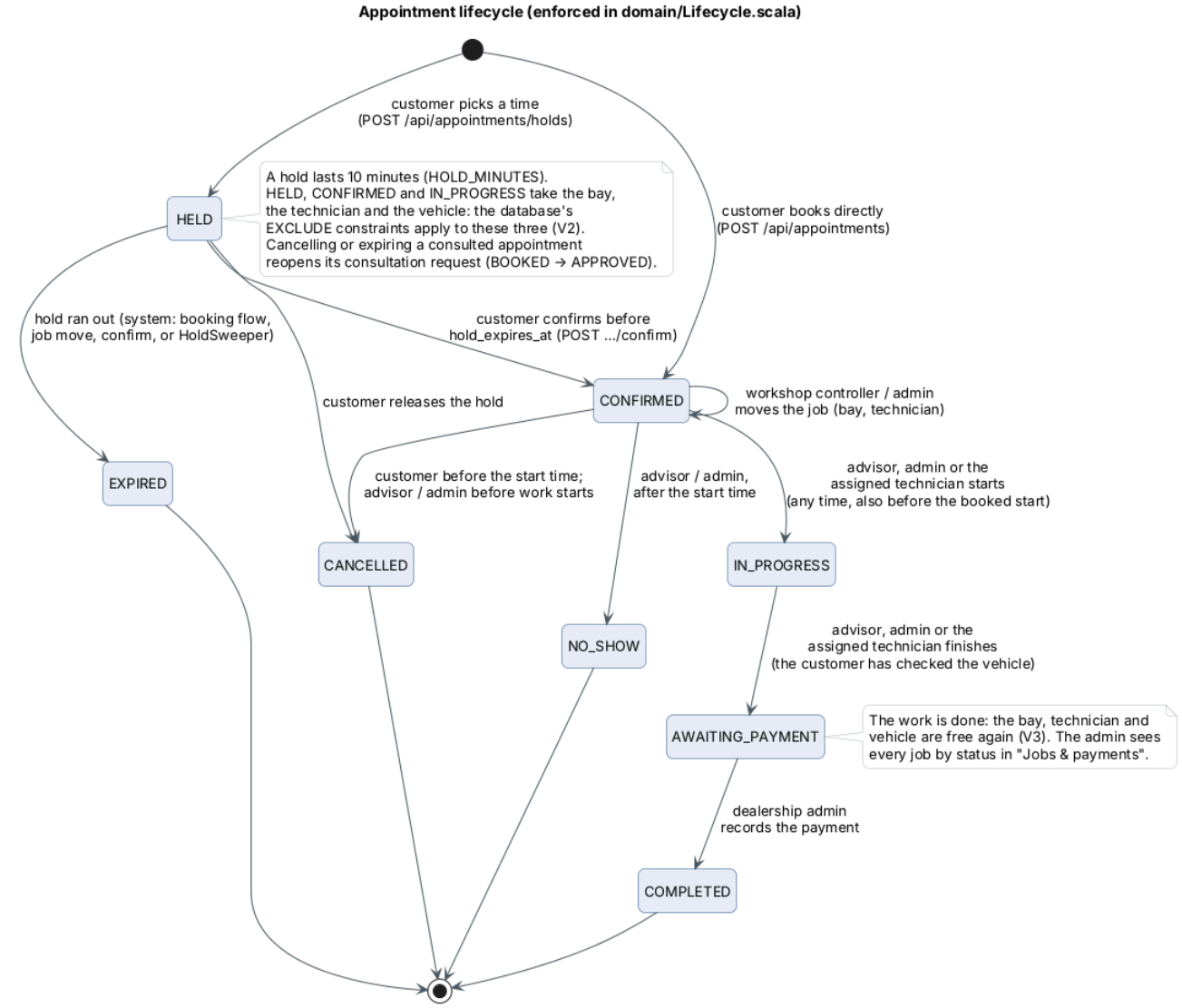


Solid: implemented Dashed grey: future work
The dealership admin can do everything a service advisor can.
Moving a job and issuing parts are checked by database constraints (no overlap; stock never below zero).

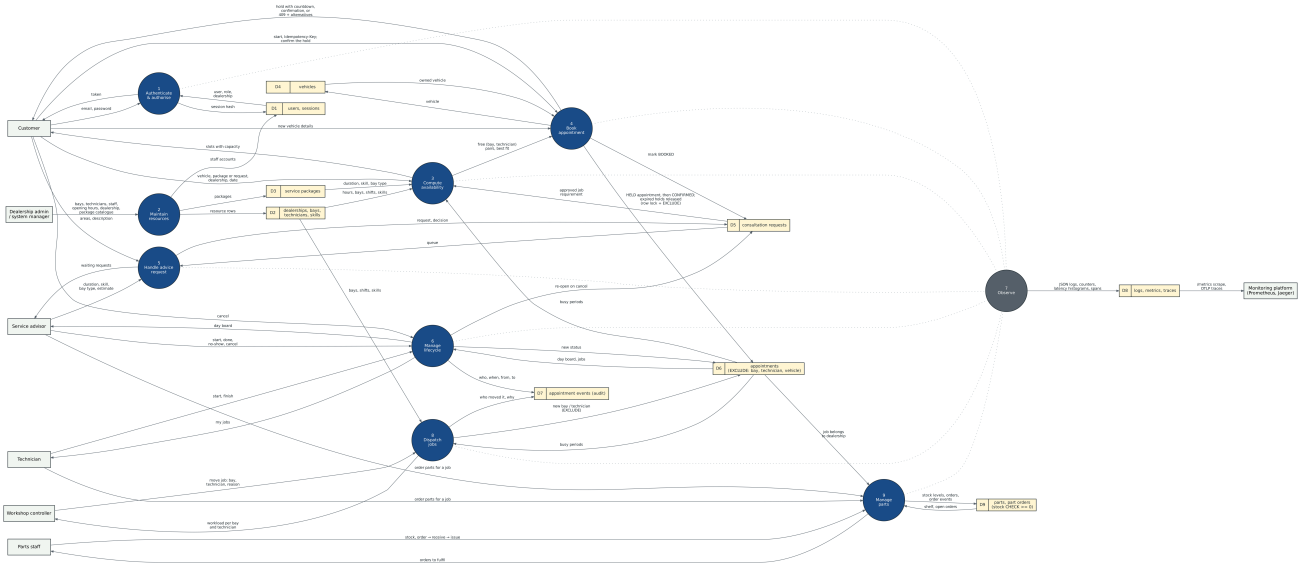

Activity — customer books a maintenance package (UC04, with UC12, UC20, UC21, UC22)
Customer
Web app
API
PostgreSQL
Has a vehicle?
yes
no
Add a vehicle
Choose the vehicle
Choose a package (only packages for this vehicle type)
Choose a dealership
Check the dealership has a bay of the package's type and a technician with its skill
Can this dealership do the job?
no: "cannot do this job here"
yes
Compute the day's start times: a free bay AND a qualified technician on shift for the whole job, inside opening hours, after lead time
Open the first day that still has a free time, refresh availability every 30 s
Pick a start time, add a note, press Hold this time and review
POST /api/appointments/holds with an Idempotency-Key (new key per distinct attempt)
Same key already booked?
yes
Return the original appointment (200, Idempotent-Replayed)
no
Start time valid? open day, on the grid, inside hours, ≥ lead time, ≤ max days ahead
no
422 with the reason
yes
Transaction: read free bays and technicians, release expired holds in the way, allocate best fit (fewest skills, least loaded)
A free pair?
no
INSERT appointment as HELD (hold_expires_at = now + 10 min)
EXCLUDE constraints reject any overlap of bay, technician or vehicle
Vehicle overlap → 409 VEHICLE_ALREADY_BOOKED (no retry: the customer must pick another time)
Lost the race on the bay or technician, and fewer than 5 attempts?
yes: retry with fresh data
no
Inserted?
yes
Commit, record HELD in the audit trail, log booking.held, count holds(created)
no
409 SLOT_UNAVAILABLE with the 3 nearest free times, log booking.slot_unavailable
Show "taken a moment ago" and the alternatives as buttons
Held?
no: pick another time or one of the alternatives
yes
Show the review screen with a countdown until hold_expires_at
Press Confirm booking
POST /api/appointments/:id/confirm (row lock on the appointment)
Still HELD and before hold_expires_at?
yes
HELD → CONFIRMED, audit event, log booking.hold_confirmed
no: ran out
HELD → EXPIRED, slot freed, 409 HOLD_EXPIRED
"Your hold ran out", offer Try to hold it again
Show the reference, technician and ticket
Optionally add to calendar (.ics)

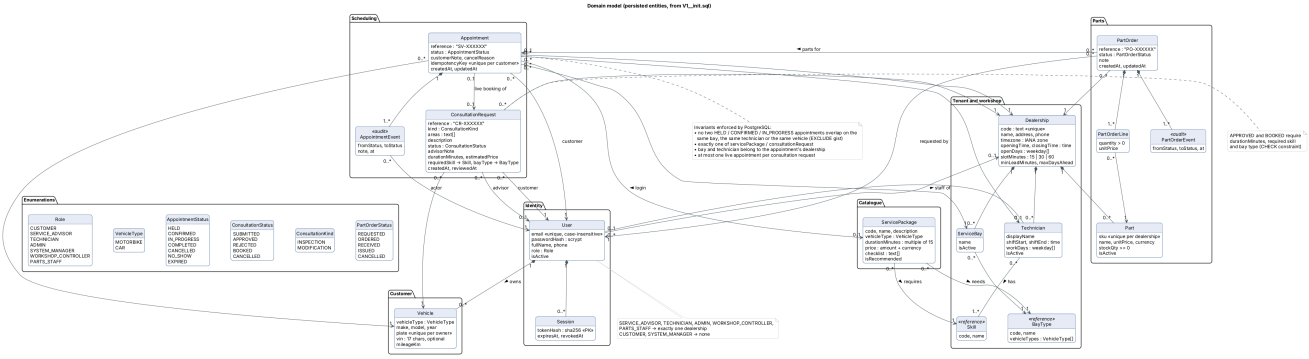

Activity — inspection on request / customisation (UC05, UC06, UC31, UC07)
Customer
Service advisor
System
Choose "Check something specific" or "Customise my vehicle"
Pick the vehicle and the areas, describe it in own words, pick a dealership
Send to an advisor
Store the request: SUBMITTED, log consultation.submitted
Open the "Waiting for advice" queue
Read the areas and the description
Can the workshop do it?
no
yes
Decline with a message to the customer
Set time needed, skill, bay type, estimate and a message, then approve
The server checks the bay type fits the vehicle and the skill exists; if not → 422 and the advisor corrects it.
REJECTED
APPROVED, log consultation.reviewed
Read the advisor's reason
See the reply on Home and on Requests (duration, estimate, advisor's note)
Book it?
withdraw
pick a time
CANCELLED
Pick a start time — only times with a bay of the approved type and a technician with the approved skill
Already booked by another tab or device → 409 CONSULTATION_NOT_BOOKABLE
Still APPROVED and the slot still free?
slot taken: offer alternatives
yes
In one transaction: appointment HELD for 10 minutes, request BOOKED
Review and press Confirm booking
Confirmed before the hold ran out?
yes
no
HELD → CONFIRMED
HELD → EXPIRED, request back to APPROVED so another time can be picked
Cancels before the start?
no
yes
Appointment CANCELLED, request back to APPROVED so another time can be picked

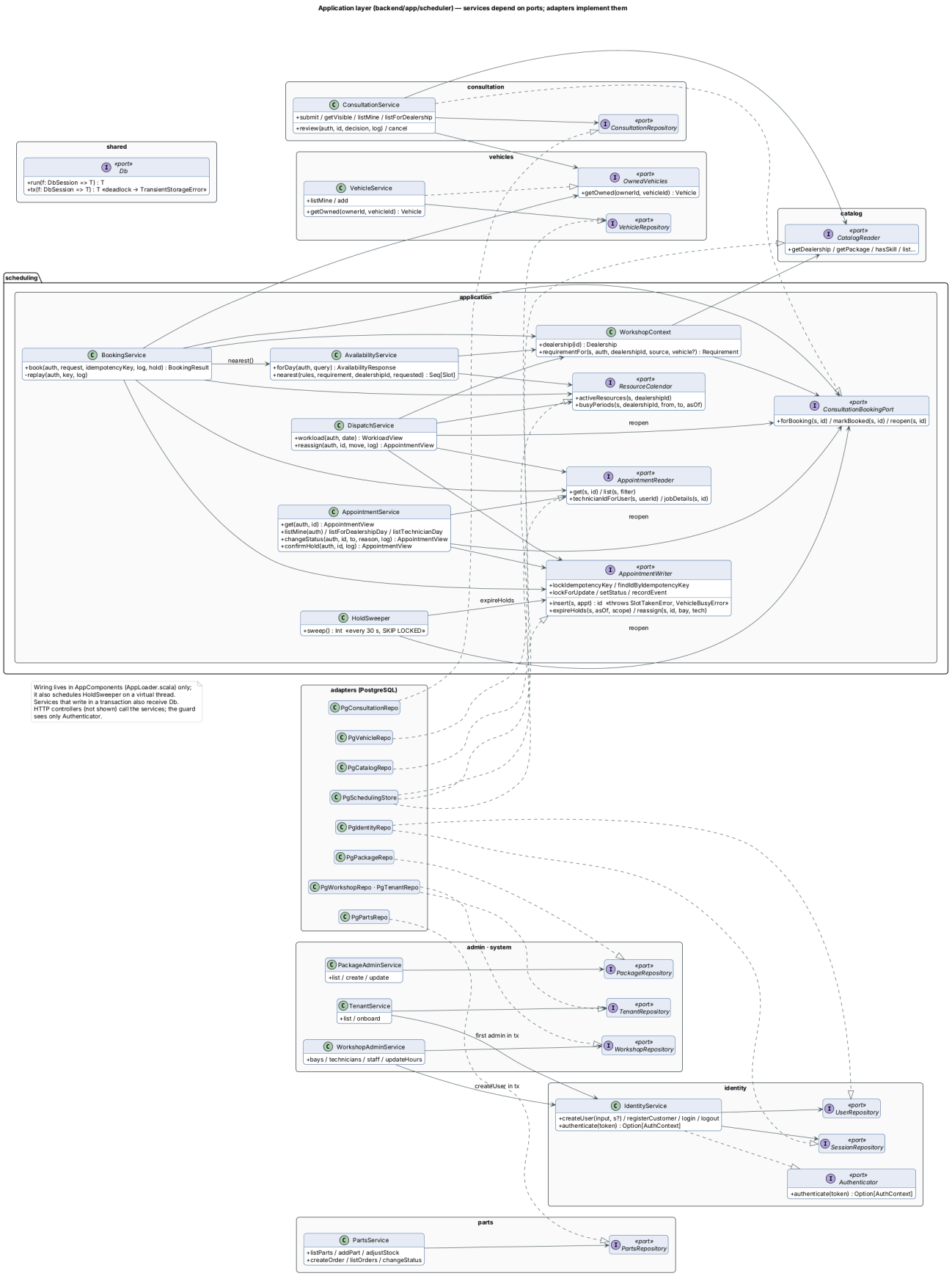

Activity — the workshop day (UC30, UC32–UC36, UC40–UC42)
Workshop controller
Service adviser
Technician
Dealership admin
System
Open the day's workload (booked minutes per bay and technician)
A technician is off or a bay is down?
yes
no
Move the affected jobs to another bay or technician, with a reason
Check the bay or technician that changes: active, right bay type, has the skill, on shift for the whole job; EXCLUDE has the last word (bay → 409 RESOURCE_BUSY); audit row "Moved ..."
Open the workshop day board (bays as rows, jobs placed by time)
"Has the vehicle arrived?" is the adviser's call at the desk. A CONFIRMED job may be started on any day, also before the booked one; the vehicle can arrive early (decided 27 Oct 2026, code review 07).
Select a booking
Has the vehicle arrived?
yes
no
Hand the vehicle to the assigned technician
Start time passed and the customer did not come?
yes
no, the booking must be dropped
Mark no-show
Cancel the booking (before work starts)
Open "My jobs": read the service, the checklist and the customer's note
CONFIRMED → NO_SHOW (bay and technician freed)
CONFIRMED → CANCELLED (slot freed, a linked request re-opens)
Parts needed?
yes
no
Order parts for the job (parts staff order, receive and issue them)
Start job
The advisor can also start it from the board.
CONFIRMED → IN_PROGRESS
Do the work
Show the customer the vehicle
Customer agrees: finish job
The advisor can also finish it from the board.
IN_PROGRESS → AWAITING_PAYMENT (bay and technician freed, the customer sees "Awaiting payment")
Customer pays at the desk
Jobs & payments: "Payment received"
No bank integration in the demo: the admin records the payment by hand. Only the admin may close a job (slice 403).
AWAITING_PAYMENT → COMPLETED (the customer sees "Finished")
Write the audit row (appointment_events), log appointment.status_changed, count the metric
Any other move (e.g. COMPLETED → IN_PROGRESS) → 409; a technician acting on someone else's job → 403. Nothing changes.



Data flow diagram, level 1: inside the Unified Service Scheduler




Domain model (persisted entities, from V1__init.sql)
Scheduling
Appointment
reference : "SV-XXXXXX"
status : AppointmentStatus
customerNote, cancelReason
createdAt, updatedAt : Instant per customer
createdAt(), updatedAt()
1
0..*
live booking of
0..1
0..*
ConsultationRequest
reference : "CR-XXXXXX"
kind : ConsultationKind
title, description
status : ConsultationStatus
advisorNote
durationInMinutes, estimatedPrice
requestedAt + limit, bayType + BayType
createdAt(), reviewedAt()
1..*
0..*
equalityAppointmentEvent
fromStatus, toStatus
note, at
0..*
0..*
0..*
0..*
0..*
0..*
pairs for
customer
requested by
actor
advisor
customer
login
staff of
Invariants enforced by PostgreSQL:
• no two HELD / CONFIRMED / INLPROGRESS appointments overlap on the same bay, the same technician or the same vehicle (EXCLUDE gist)
• exactly one of servicePackage / consultationRequest
• bay and technician belong to the appointment's dealership
• at most one live appointment per consultation request
Parts
PartOrder
reference : "PO-XXXXXX"
status : PartOrderStatus
note
createdAt, updatedAt
1..*
0..*
PartOrderLine
quantity > 0
unitPrice
1..*
equalityPartOrderEvent
fromStatus, toStatus, at
0..*
Part
sku + unique per dealership
name, unitPrice, currency
stockQty >= 0
isActive
APPROVED and BOOKED require durationInMinutes, required skill and bay type (CHECK constraint)
Tenant and workshops
Dealership
code : text +unique
name, addressLine
timezone : IANA zone
openingTime, closingTime : time
openDays : weekday[]
slotMinutes : 15 | 30 | 60
minLeadMinutes, maxDaysAhead
1..1
0..*
0..1
0..*
ServiceBay
name
isActive
1..0..*
0..*
Technician
displayName
shiftStart, shiftEnd : time
workDays : weekday[]
isActive
0..*
0..*
equalityServiceSkill
code, name
1..*
0..*
equalityBayType
code, name
vehicleTypes : VehicleType[]
1..1
0..*
Identity
User
email +unique, case-insensitive
passwordHash : scrypt
fullName, phone
role : Role
isActive
1
1
1
0..*
Session
tokenHash : sha256 +PK
expiresAt, revokedAt
SERVICE_ADVISOR, TECHNICIAN, ADMIN, WORKSHOP_CONTROLLER, PARTS_STAFF -> exactly one dealership
CUSTOMER, SYSTEM_MANAGER -> none
Catalogue
ServicePackage
code, name, description
vehicleType : VehicleType
durationMinutes : multiple of 15
price : amount + currency
checklist : text[]
isRecommended
0..1
0..*
requires
0..*
needs
has
Customers
Vehicle
vehicleType : VehicleType
make, model, year
plate +unique per owner+
vin : 17 chars, optional
isArsagnitos
1
0..*
owns
Enumerations
Role
CUSTOMER
SERVICE_ADVISOR
TECHNICIAN
ADMIN
SYSTEM_MANAGER
WORKSHOP_CONTROLLER
PARTS_STAFF
VehicleType
MOTORBIKE
CAR
AppointmentStatus
HELD
CONFIRMED
INLPROGRESS
COMPLETED
CANCELLED
NO_SHOW
EXPIRED
ConsultationStatus
SUBMITTED
UNDERREVIEW
REJECTED
BOOKED
CANCELLED
ConsultationKind
INSPECTION
MODIFICATION
PartOrderStatus
REQUESTED
RECEIVED
ISSUED
CANCELLED


Application layer (backend/app/scheduler) — services depend on ports; adapters implement them
shared
«port»
Db
+run(f: DbSession => T) : T
+tx(f: DbSession => T) : T «deadlock → TransientStorageError»
consultation
ConsultationService
+submit / getVisible / listMine / listForDealership
+review(auth, id, decision, log) / cancel
«port»
ConsultationRepository
vehicles
VehicleService
+listMine / add
+getOwned(ownerId, vehicleId) : Vehicle
«port»
OwnedVehicles
+getOwned(ownerId, vehicleId) : Vehicle
«port»
VehicleRepository
catalog
«port»
CatalogReader
+getDealership / getPackage / hasSkill / list...
scheduling
application
BookingService
+book(auth, request, idempotencyKey, log, hold) : BookingResult
-replay(auth, key, log)
nearest()
AvailabilityService
+forDay(auth, query) : AvailabilityResponse
+nearest(rules, requirement, dealershipId, requested) : Seq[Slot]
WorkshopContext
+dealership(id) : Dealership
+requirementFor(s, auth, dealershipId, source, vehicle?) : Requirement
«port»
ResourceCalendar
+activeResources(s, dealershipId)
+busyPeriods(s, dealershipId, from, to, asOf)
«port»
ConsultationBookingPort
+forBooking(s, id) / markBooked(s, id) / reopen(s, id)
reopen
DispatchService
+workload(auth, date) : WorkloadView
+reassign(auth, id, move, log) : AppointmentView
«port»
AppointmentReader
+get(s, id) / list(s, filter)
+technicianIdForUser(s, userId) / jobDetails(s, id)
reopen
AppointmentService
+get(auth, id) : AppointmentView
+listMine(auth) / listForDealershipDay / listTechnicianDay
+changeStatus(auth, id, to, reason, log) : AppointmentView
+confirmHold(auth, id, log) : AppointmentView
«port»
AppointmentWriter
+lockIdempotencyKey / findIdByIdempotencyKey
+lockForUpdate / setStatus / recordEvent
+insert(s, appt) : id «throws SlotTakenError, VehicleBusyError»
+expireHolds(s, asOf, scope) / reassign(s, id, bay, tech)
reopen
expireHolds
HoldSweeper
+sweep() : Int «every 30 s, SKIP LOCKED»
Wiring lives in AppComponents (AppLoader.scala) only;
it also schedules HoldSweeper on a virtual thread.
Services that write in a transaction also receive Db.
HTTP controllers (not shown) call the services; the guard
sees only Authenticator.
adapters (PostgreSQL)
PgConsultationRepo
PgVehicleRepo
PgCatalogRepo
PgSchedulingStore
PgIdentityRepo
PgPackageRepo
PgWorkshopRepo · PgTenantRepo
PgPartsRepo
admin · system
PackageAdminService
+list / create / update
«port»
PackageRepository
TenantService
+list / onboard
«port»
TenantRepository
first admin in tx
WorkshopAdminService
+bays / technicians / staff / updateHours
«port»
WorkshopRepository
createUser in tx
identity
IdentityService
+createUser(input, s?) / registerCustomer / login / logout
+authenticate(token) : Option[AuthContext]
«port»
UserRepository
«port»
SessionRepository
«port»
Authenticator
+authenticate(token) : Option[AuthContext]
parts
PartsService
+listParts / addPart / adjustStock
+createOrder / listOrders / changeStatus
«port»
PartsRepository

10. Technologies

Layer	Technology	Version	Why
Language	Scala	3.3.6 LTS (also 3.9.0, 3.10.0)	Strong types and enums for the domain
Runtime	Java (JDK)	21+	Virtual threads keep blocking JDBC code simple under load
Web framework	Play Framework (Pekko HTTP)	3.0.12	Routing, JSON, non-blocking HTTP server; compile-time wiring
Build	sbt	1.10.7	Standard Scala build; the launcher downloads the right version
Database	PostgreSQL	15+ (17 embedded)	Range types and exclusion constraints enforce BR-01 in the database
Migrations	Flyway	11.20.3	Versioned, locked migrations
SQL access	Anorm + HikariCP	2.7.0 + 6.3.0	Plain SQL with typed parameters; fast pool
Embedded database	Zonky embedded-postgres	2.1.0, PostgreSQL 17.6	`sbt run` and tests without Docker
JSON	play-json, Jackson	2.19.1 (pinned)	Validation with per-field errors
Logging	Logback + logstash-logback-encoder	8.0	One JSON object per log line
Metrics	Prometheus Java client	1.10.0	Standard `/metrics` endpoint
Tracing	OpenTelemetry SDK, OTLP exporter (Jaeger)	see `build.sbt`	Request, booking-attempt and SQL spans; W3C `traceparent` is continued
End-to-end tests	Selenium WebDriver + ScalaTest (`e2e/`)	see `e2e/build.sbt`	Drives Chrome through the real web app on every pull request
API docs	OpenAPI + Swagger UI	3.1 + 5.17.14	Contract served at `/docs`
Tests	ScalaTest (scalatestplus-play 7.0.2) + ScalaCheck	3.2.19 + 1.18.1	Readable specs; property-based tests for scheduling invariants
Frontend	React, React Router, TanStack Query	19, 7, 5	SPA with cached, auto-refreshing queries
Frontend build	Vite, TypeScript	8, 6	Fast dev server with API proxy
Dates	java.time (API), Luxon (UI)	—	Explicit time-zone arithmetic
Diagrams	PlantUML, Graphviz	1.2025.4	Diagrams as text, kept with the code

11. Installation guide

11.1 Prerequisites

Tool	Version	Check with
JDK	21 or newer (Temurin recommended)	`java -version`
sbt	1.10 or newer	`sbt --version`

Tool	Version	Check with
Node.js	22 or newer (includes npm)	`node -v`
Git (optional)	any	`git --version`

Docker is **not** needed: the API starts its own PostgreSQL.

11.2 Install the tools

macOS: `brew install openjdk@21 sbt node`.

Linux (Debian / Ubuntu): install Temurin 21 and sbt from their official apt repositories, and Node.js 22 from NodeSource or `nvm`.

Windows with administrator rights: install the Temurin 21 MSI, the sbt MSI and the Node.js LTS MSI, then open a new terminal.

Windows without administrator rights (for example a school or company PC):

1. Download the **Temurin JDK 21 .zip** (not the MSI) from adoptium.net and unzip it to `C:\Users\<you>\tools\jdk-21`.
2. Download **sbt-1.10.x.zip** from scala-sbt.org and unzip it to `C:\Users\<you>\tools\sbt`.
3. Download the **Node.js .zip** for Windows and unzip it to `C:\Users\<you>\tools\node`.
4. Add them to *your user* PATH in PowerShell (no admin needed), then open a new terminal:

```
        $t = "$env:USERPROFILE\tools"
        [Environment]::SetEnvironmentVariable("JAVA_HOME", "$t\jdk-21", "User")
        $p = [Environment]::GetEnvironmentVariable("Path", "User")
        [Environment]::SetEnvironmentVariable("Path", "$p;$t\jdk-
21\bin;$t\sbt\bin;$t\node", "User")
```

5. Check: `java -version`, `sbt --version`, `node -v`.

11.3 Get the code and run it

```
        # unzip service-scheduler.zip (or git clone), then from the project root:
        npm run setup     # once: installs the web app's dependencies
        npm run dev       # starts the API (sbt run + embedded PostgreSQL) on :8080 and the
web app on :5173
```

`npm run dev` checks that `java` and `sbt` are on the PATH and that `npm run setup` has been run, and says what is missing. The first start downloads dependencies and compiles the Scala code; it takes one to three minutes. It is ready when the log shows `server.ready`.

Open	For
http://localhost:5173	The web app (demo accounts on the sign-in page, password `Demo@2026`)
http://localhost:8080/docs	Swagger UI
http://localhost:8080/health	Health check

To run only the API: `cd backend && sbt run`.

11.4 Common tasks

Task	How
Start with fresh demo data	Stop the app, delete `.data/pg-54330`, start again
Run the tests	`npm test` (95 tests), `npm run test:load` (1000-request bursts)
Use another API port, for example 8081	Either run `cd backend && sbt "run 8081"`, or change `PlayKeys.playDefaultPort` in `backend/build.sbt` and `server.http.port` in `backend/conf/application.conf`. In both cases also change the proxy targets in `frontend/vite.config.ts` to `http://localhost:8081`
Use another web app port	Change `server.port` in `frontend/vite.config.ts`, and add the new origin to `CORS_ORIGIN` if the browser calls the API directly
Use your own PostgreSQL	`docker compose up -d` (or any PostgreSQL 15+), then set `DATABASE_URL=postgres://scheduler:scheduler@localhost:5432/scheduler` before `npm run dev`
Readable or JSON logs	`LOG_FORMAT=pretty` (default in `npm run dev`) or `LOG_FORMAT=json`
Use Scala 3.9 or 3.10	Change `scalaVersion` in `backend/build.sbt`; sbt downloads the compiler

11.5 Production-style package

```
cd backend
sbt stage
DATABASE_URL=postgres://user:pass@db-host:5432/scheduler \
APPLICATION_SECRET=<a long random string> \
CORS_ORIGIN=https://booking.example.com \
SEED_DEMO_DATA=false \
target/universal/stage/bin/service-scheduler
```

Build the web app with `npm --prefix frontend run build` and serve `frontend/dist` from any static web server that forwards `/api` to the API. Run several API instances behind a load balancer if needed (NFR-03); Flyway locks migrations so two instances never migrate at once.

11.6 Configuration

Variable	Default	Meaning
`DATABASE_URL`	empty → embedded PostgreSQL	`postgres://…` or `jdbc:postgresql://…`
`EMBEDDED_PG_PORT`	54330	Port of the embedded PostgreSQL
`DATA_DIR`	`../.data` (project root)	Where the embedded PostgreSQL keeps its files
`PORT`	8080	HTTP port of the packaged server
`CORS_ORIGIN`	`http://localhost:5173`	Comma-separated allowed browser origins
`SEED_DEMO_DATA`	true	Load demo data into an empty database
`LOG_FORMAT`, `LOG_LEVEL`	json, INFO	`pretty` for readable local lines

Variable	Default	Meaning
`SLOW_QUERY_MS`	200	Log queries slower than this
`SESSION_TTL_HOURS`	12	Sign-in lifetime
`DB_POOL_SIZE`	20	Connections per API instance
`APPLICATION_SECRET`	development value	Required by Play in production
`HOLD_MINUTES`	10	How long a hold keeps its time before it must be confirmed
`HOLD_SWEEP_SECONDS`	30	How often each instance releases lapsed holds (0 turns the sweeper off)
`OTEL_EXPORTER_OTLP_ENDPOINT`	empty → no export	Where to send traces, e.g. `http://jaeger:4317`

11.7 Troubleshooting

Symptom	Cause	Fix
`'sbt' is not recognized` / `sbt: command not found`	sbt is not installed or not on the PATH	Section 11.2; open a new terminal after changing the PATH
`'vite' is not recognized`	The web app's dependencies are not installed	`npm run setup`
`spawn EINVAL` on Windows with Node 24	An old copy of the project (0.1.0)	Use version 0.2.0 or later; `package.json` shows the version
`Address already in use` / bind failure on 8080	Another program uses port 8080	Stop it, or move the API to another port (section 11.4)
The sign-in page says 404 or cannot reach the server	The API is not running or is still compiling	Wait for `server.ready`; check http://localhost:8080/health
Embedded PostgreSQL refuses to start on Linux as root	PostgreSQL never runs as root	Run as a normal user, or set `DATABASE_URL`
The first start is slow	sbt downloads dependencies and compiles	Only the first time; later starts take seconds

12. Out of scope and future work

Rescheduling (today: cancel and rebook), staff booking on behalf of a phone or walk-in customer, editing vehicles and technicians, breaks and holidays in shifts, notifications (SMS, Zalo, email), jobs needing several technicians, rate limiting on sign-in.

Built since version 1.0 of this document (no longer future work): the 10-minute hold before confirming, parts stock and orders, the workshop-controller and parts-staff roles, opening hours in the UI, the service package catalogue, OpenTelemetry tracing, and a CI/CD pipeline with Selenium end-to-end tests.